# LAPORAN MINI PROJECT STUDI INDEPENDEN: PERANCANGAN DATA WAREHOUSE DAN DATA MART CHINOOK STORE UNTUK ANALISIS PENJUALAN MUSIK GLOBAL

---

## A. TEAM AND SELECTED DATABASE

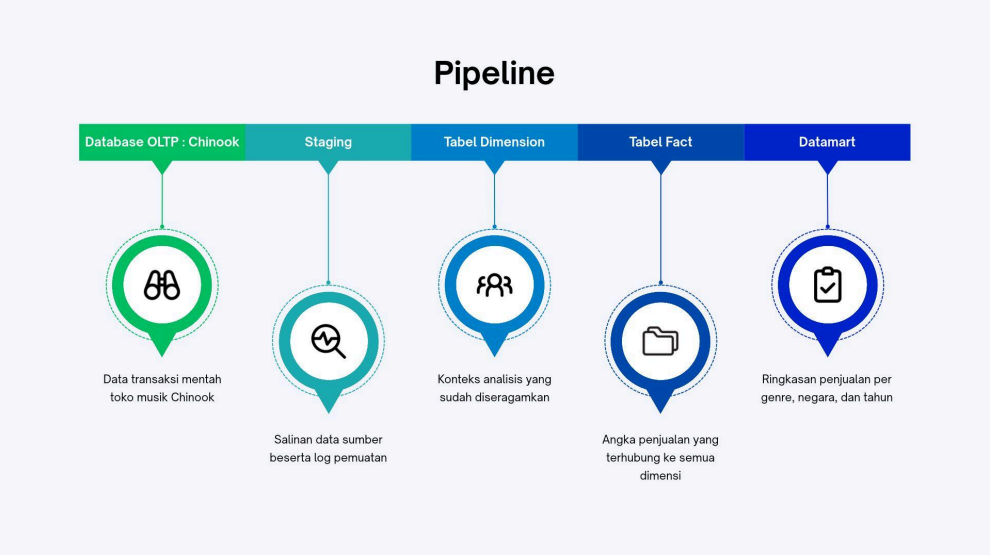
### 1. Anggota Team

1. Muhammad Ramadhan Albaary Putra (24031554161)/ S1 Sains Data
2. Muslim Fazlur Rohman (24031554154)/ S1 Sains Data
3. Naufal Muzaki (2403155061)/ S1 Sains Data
4. Eka Aliya (24031554012)/ S1 Sains Data
5. Maria Agatha Metri Tetrajati (24031554189)/ S1 Sains Data

**Mentor Personal:** Ryan Firmansyah

### 2. Database (Chinook Database) dan Pipeline Proyek

Pada *mini project* ini, kami memilih *dataset Chinook Store* sebagai *database* utama karena struktur datanya menggambarkan sistem operasional toko musik digital yang nyata dan lengkap. Di dalamnya, data transaksi, pelanggan, staf, hingga katalog lagu tersusun rapi. Relasi antar tabel pada sistem transaksi *Chinook Store* ini sangat pas untuk dijadikan bahan latihan pemodelan *Star Schema*. Tabel- tabel yang awalnya terpisah bisa kita gabung dan rapikan (denormalisasi) menjadi tabel-tabel dimensi serta satu tabel fakta. Selain itu, datanya juga sangat menarik untuk dianalisis, kita bisa melihat genre lagu apa yang paling laku, tren pembelian di berbagai negara, sampai perkembangan pendapatan bisnis tahun per tahun.




**Gambar 1.** Pipeline Alur Pengerjaan Proyek

Berdasarkan gambar pipeline tersebut, berikut langkah-langkah pengerjaan yang akan kami lakukan:

- Data Transaksi Mentah Toko Musik Chinook : dari data transaksi mentah ini, kami melakukan eksplorasi dataset, mulai dari cek struktur per tabel dan jumlah barisnya, serta mengecek terkait adanya *missing value* dan duplikasi data.
- Proses Staging: langkah selanjutnya, kami mulai melakukan proses staging. Di sini, kami menyiapkan database (*staging_chinook*) dengan dua *schema* terpisah, yaitu *staging* (untuk menyimpan salinan tabel operasional) dan *log* (untuk mencatat histori pemrosesan, seperti *staging_log* dan *log_table_entry*). Pemuatan data dilakukan secara otomatis dengan menggunakan Pentaho melalui kombinasi transformasi dan *job*. Transformasi ini nantinya berisi step-step yang akan kami lakukan terhadap data, mulai dari memanggil data, menyeleksi tabel sumber dan membandingkannya dengan yang ada di *schema log* agar tabel yang sudah diproses dapat dilewati secara otomatis, sampai proses ekstraksi data ke *schema staging*. Seluruh alur pada tahap transformasi ini nantinya akan diproses di bagian *Job*, proses eksekusinya mulai dari pemuatan per tabel, menghitung jumlah baris target, dan memvalidasi kesesuaian data. Jika jumlah baris cocok maka bagian status akan diisi '*succes*' dan nama tabel akan dicatat ke bagian *log_table_entry*.
- Proses *Dimension* tabel: setelah proses *staging*, kami membuat beberapa *dimension* tabel dari data hasil staging dan disesuaikan dengan kebutuhan analisis kami kedepannya. Hasil tabel dimensi ini nantinya akan disimpan ke database baru (*dwh_chinook*) pada s*chema dimension.*
- Proses *fact* tabel: Setelah seluruh dimension table terbentuk, tahap selanjutnya adalah membangun tabel fakta (*fact_sales*) yang menghubungkan data penjualan dengan seluruh *dimension table*. Data transaksi dari *staging* dicocokkan dengan masing-masing dimension table untuk memperoleh *Surrogate Key*, kemudian disimpan dalam database (*dwh_chinook*) pada schema *fact_table* sebagai dasar analisis penjualan.
- Proses data mart: Data dari tabel (*fact_sales*) digabungkan dengan tabel dimensi, yaitu dimensi t*rack, customer*, dan *date*, untuk menghasilkan informasi penjualan berdasarkan genre, negara, dan tahun. Data tersebut kemudian diagregasi untuk memperoleh total pendapatan, jumlah unit terjual, jumlah pelanggan, serta peringkat pendapatan. Hasil akhirnya disimpan dalam tabel *dm_sales_genre_country* dan diekspor ke format CSV.

## B. INITIAL DATA EXPLORATION

### 1. Cek struktur dan jumlah data

Pada database chinook, kami melakukan pemeriksaan terhadap struktur kolom, jumlah baris, dan hubungan antar tabel. terdapat 11 tabel pada *schema public* dengan rincian berikut:

**Tabel 1.** Jumlah Baris Database Chinook

| Tabel | Jumlah Baris |
|---|---|
| album | 347 |
| artist | 275 |
| customer | 59 |
| employee | 8 |
| genre | 25 |
| invoice | 412 |
| invoice_line | 2240 |
| media_type | 5 |
| playlist | 18 |
| playlist_track | 8715 |
| track | 3503 |

Pada data transaksi, tabel *invoice* terhubung dengan customer melalui kolom *customer_id*, lalu terhubung dengan *invoice_line* melalui *invoice_id*. Tabel *invoice_line* juga terhubung dengan *track* melalui *track_id*. Sementara itu, informasi produk musik tersebar pada tabel *track*, album, artist, genre, dan *media_type.* Kami juga memeriksa hubungan pelanggan dengan pegawai melalui kolom *support_rep_id*, serta hubungan playlist dengan *track* melalui tabel *playlist_track.*

### 2. Cek Missing Value dan Duplikasi

hasil pemeriksaan menunjukkan adanya nilai *null* pada beberapa kolom berikut:

**Tabel 2**. Hasil Pemeriksaan Missing Value

| Tabel | Kolom | Jumlah NULL |
|---|---|---|
| customer | company | 49 |
| customer | state | 29 |

| | | |
|---|---|---|
| customer | postal_code | 4 |
| customer | phone | 1 |
| customer | fax | 47 |
| employee | reports_to | 1 |
| invoice | billing_state | 202 |
| invoice | billing_postal_code | 28 |
| track | composer | 977 |

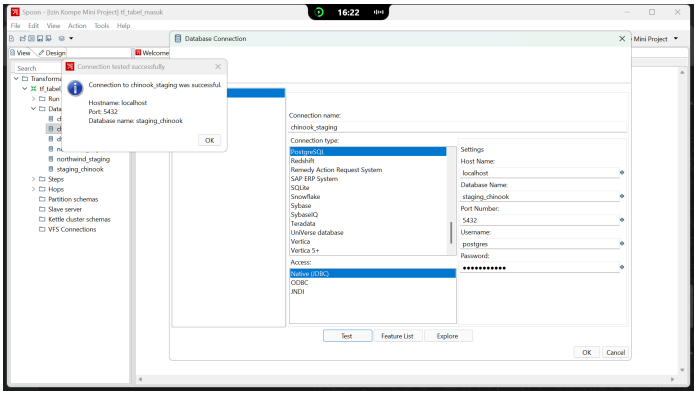
Selain itu, kami memeriksa duplikasi data di setiap tabel. Dari hasil pemeriksaan data backup, tidak ditemukan duplikasi pada kunci tersebut. Nilai null yang ditemukan masih dipertahankan saat pemindahan data ke *staging*. Penanganan nilai kosong dan standarisasi atribut dilakukan pada proses pembentukan tabel dimensi.

## C. STAGING PROCESS

### 1. Database Connection dan Staging Setup

Kami mengatur dua koneksi di Pentaho, yaitu *chinook_oltp* untuk membaca data sumber dan *chinook_staging* untuk menyimpan hasil pemindahan data ke database *staging_chinook.* Pada database tersebut, kami membagi penyimpanan menjadi dua schema, yaitu:

- *staging*: menyimpan salinan tabel dari database operasional
- *log*: menyimpan catatan pemrosesan pada tabel *staging_log* dan *log_table_entry*




**Gambar 2.** Database Connection and Staging Setup

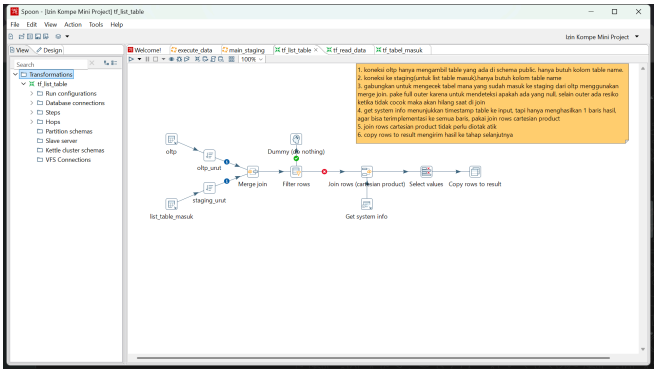
### 2. Staging Transformation

Pada proses transformasi, kami menggunakan *tf_list_table.ktr* untuk menyeleksi tabel yang akan diproses dan *tf_read_data.ktr* untuk memindahkan data ke staging. Kami melakukan serangkaian proses berikut:

- **List dan Sorting Data Tabel**

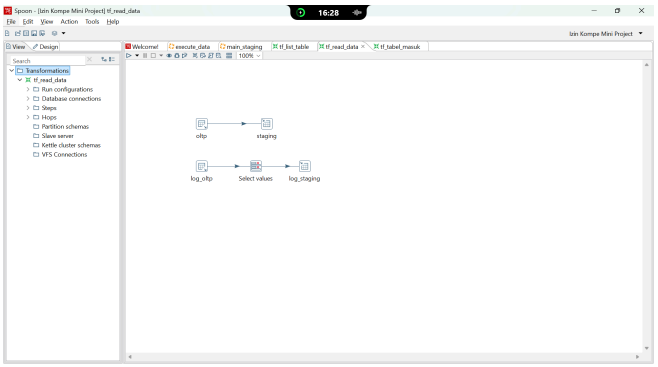
Pada transformasi *tf_list_table*, step oltp membaca daftar tabel sumber dari *information_schema.tables* dengan filter *table_schema = 'public'*. Sementara itu, step list_table_masuk membaca daftar tabel yang sudah tercatat pada *log.log_table_entry.*

- **Join dan Filter Tabel**
  Kedua daftar tabel digabungkan melalui step *Merge Join* menggunakan *Full Outer Join,* lalu diperiksa di *Filter rows*. Tabel yang tercatat diarahkan ke *Dummy (do nothing)*, sedangkan tabel yang belum tercatat diteruskan ke proses berikutnya.
- Penambahan Waktu Awal Proses
  Kami menambahkan field mulai melalui step *Get system info*. *Field* tersebut digabungkan dengan daftar tabel menggunakan *Join rows (cartesian product)*. Lalu, step *Select values* memilih *field table_name* dan mulai, lalu *Copy rows to result* meneruskannya ke job.




**Gambar 3.** Staging Transformation

- **Extract dan Load Data**
  Pada transformasi *tf_read_data* step *oltp* membaca seluruh data dari tabel. data kemudian dimasukkan melalui step table output. pada step ini kami mengaktifkan pengaturan *truncate table* dan *batch insert,* serta mengatur *commit* setiap 1000 baris
- **Source Data Logging**
  Step *log_oltp* menghitung jumlah baris sumber dan mengambil informasi *table_name, source_records, source_date*, serta *etl_date_start.* Setelah itu, *field etl_date_start* diubah menjadi tipe *timestamp* melalui *select values*. Seluruh informasi tersebut kemudian dimasukkan ke *log.staging_log* melalui step *log_staging.*

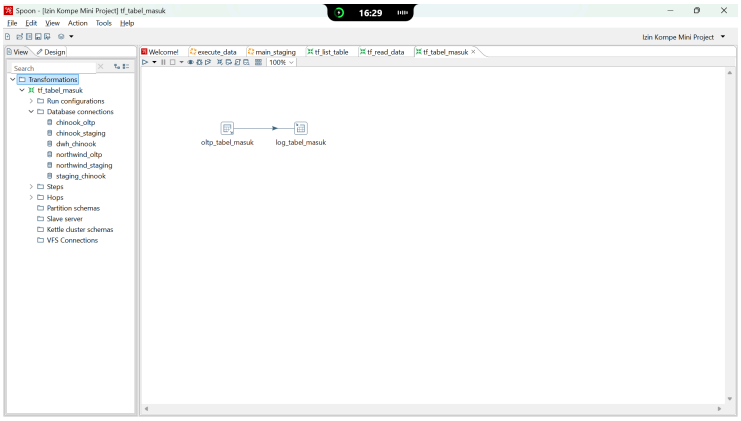



**Gambar 4.** Source Data Logging

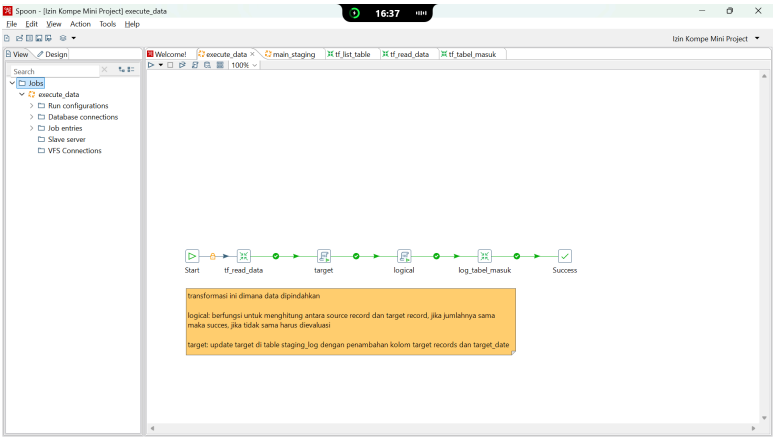
### 3. Staging Job

Kami menyusun *job execute_data.kjb* untuk menjalankan pemuatan dan validasi setiap tabel, kemudian menghubungkannya melalui *job* utama *main_staging.kjb.* Pada kedua job tersebut, kami melakukan beberapa pengaturan berikut:

- **Eksekusi Transformasi Pemuatan Data**
  Pada job *execute_data*, kami menyusun urutan *Start* → *tf_read_data* → target → *logical* → log_tabel_masuk → Success. Entry *tf_read_data* menjalankan transformasi pemuatan dengan menerima parameter *table_name* dan mulai.
- **Penghitungan Jumlah Data Target**
  Setelah pemuatan selesai, *entry SQL* target menghitung jumlah baris pada tabel. Hasilnya dimasukkan ke kolom *target_records*, sedangkan waktu pemeriksaannya dicatat pada kolom *target_date* di tabel *log.staging_log.*
- **Validasi Jumlah Data**
  Pada *entry SQL logical*, kami membandingkan nilai *source_records* dan *target_records*. Jika jumlahnya sama, kolom *logical* diisi dengan status '*succes*', sesuai penulisan pada konfigurasi. Jika jumlahnya berbeda, status diisi *'to be evaluated'*.
- **Table Logging**
  Selanjutnya, entry log_tabel_masuk menjalankan transformasi tf_tabel_masuk.ktr. Transformasi ini mengambil nama tabel dan waktu pencatatan, kemudian memasukkannya ke tabel *log.log_table_entry*.



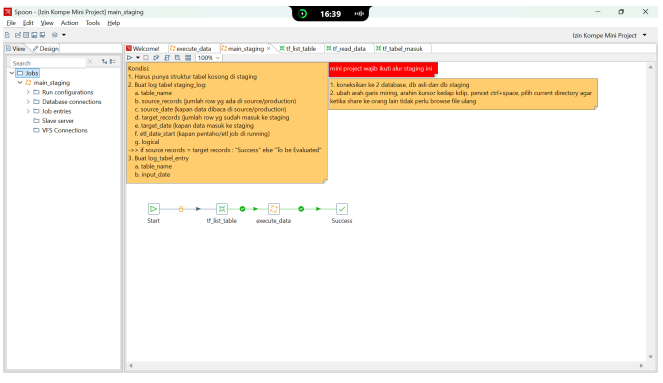

**Gambar 5.** Transformation Tabel Masuk

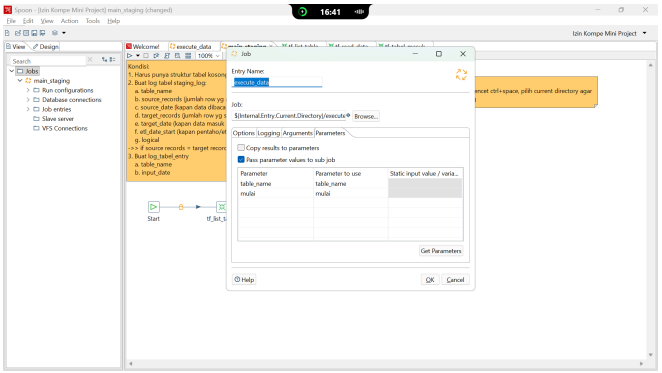



**Gambar 6.** Job Execute Data

- **Main Job Configuration**

  Pada job *main_staging,* kami menyusun urutan *Start → tf_list_table → execute_data → Success*. *Entry execute_data* diatur agar dijalankan untuk setiap baris hasil dari *tf_list_table*. *Field table_name* dan mulai diteruskan sebagai parameter untuk memproses masing-masing tabel. Ketika job utama dijalankan kembali, tabel yang sudah tercatat pada *log.log_table_entry* akan dilewati.





**Gambar 7**. Main Job Configuration

### 4. Data Loading dan Validasi

Berdasarkan hasil perbandingan data sumber dan *staging* seluruh tabel memiliki jumlah baris yang sama. Perbandingan isi baris pada kedua backup juga menunjukkan kesamaan data pada seluruh tabel tanpa memperhitungkan urutan baris.

Pada tabel *log.log_table_entry*, terdapat 11 catatan tabel yang sudah diproses. Sementara itu, tabel *log.staging_log* memuat 13 catatan karena tabel album memiliki tiga catatan pemrosesan. Seluruh catatan memiliki nilai *source_records* dan *target_records* yang sama dengan status '*succes*'.

| table_name | source_records | source_date | target_records | target_date | etl_date_start | logical |
|---|---|---|---|---|---|---|
| album | 347 | 52:37.0 | 347 | 06:00.8 | 52:35.3 | succes |
| album | 347 | 04:50.8 | 347 | 06:00.8 | 04:49.6 | succes |
| album | 347 | 06:00.6 | 347 | 06:00.8 | 05:59.5 | succes |
| artist | 275 | 06:01.5 | 275 | 06:01.7 | 05:59.5 | succes |
| customer | 59 | 06:02.4 | 59 | 06:02.5 | 05:59.5 | succes |
| employee | 8 | 06:03.1 | 8 | 06:03.3 | 05:59.5 | succes |
| genre | 25 | 06:04.0 | 25 | 06:04.4 | 05:59.5 | succes |
| invoice | 412 | 06:05.0 | 412 | 06:05.3 | 05:59.5 | succes |
| invoice_line | 2240 | 06:06.2 | 2240 | 06:06.6 | 05:59.5 | succes |
| media_type | 5 | 06:07.2 | 5 | 06:07.4 | 05:59.5 | succes |
| playlist | 18 | 06:08.1 | 18 | 06:08.2 | 05:59.5 | succes |
| playlist_track | 8715 | 06:09.2 | 8715 | 06:09.7 | 05:59.5 | succes |
| track | 3503 | 06:10.3 | 3503 | 06:10.8 | 05:59.5 | succes |

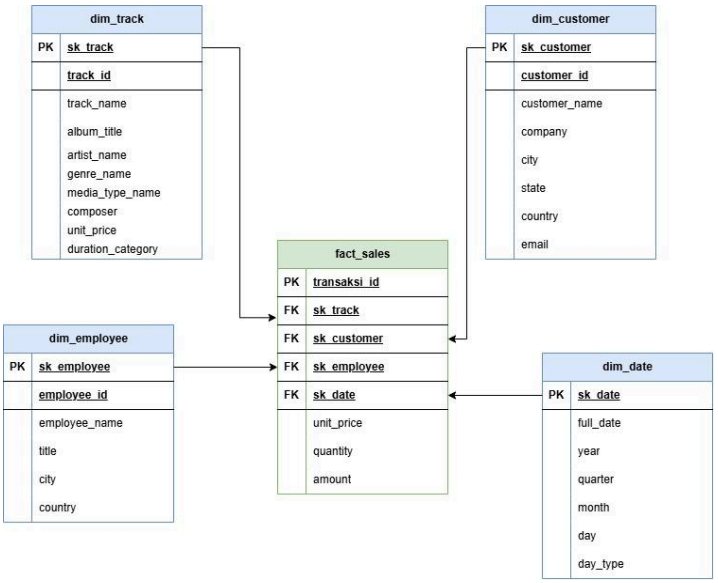
**Gambar 8.** Data Loading dan Validasi

## D. DIMENSION AND FACT TABLE DESIGN

### 1. Pembuatan Desain Star Schema dan Pembentukan Database

Pada *dataset chinook store* kami memilih pemodelan *star schema* untuk membuat struktur desain tabel dimensi dan tabel fakta. Alasan pemilihan model *star schema* ini didasarkan pada strukturnya yang sederhana dan efisien untuk kebutuhan analisis bisnis. Dengan menempatkan satu tabel fakta (*fact_sales*) ditengah dan dikelilingi oleh tabel-tabel dimensi (d*im_track, dim_customer, dim_date, dim_employee*), performa *query* untuk proses analisis akan jauh lebih cepat dan ringan karena meminimalkan proses *JOIN*

antar tabel yang kompleks jika dibandingkan dengan struktur database operasional (OLTP).



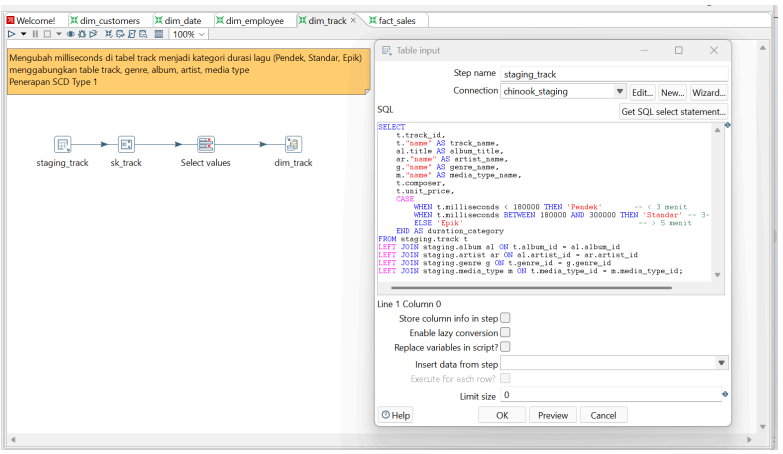

**Gambar 9.** Rancangan Star Schema dengan Draw.io

Selanjutnya, kami juga membuat database baru (*dwh_chinook*) di *PostgreSQL* sebagai tempat penyimpanan akhir dari seluruh hasil transformasi data dimensi dan tabel fakta yang diproses melalui Pentaho. Di dalam database ini kami membagi bagian *schema* menjadi dua yaitu:

- *dimension*: *schema* khusus untuk menyimpan seluruh tabel dimensi.
- *fact_table*: *schema* khusus untuk menyimpan tabel fakta

Pemisahan ini dilakukan agar struktur database menjadi lebih rapi dan terorganisir.

## 2. Perancangan dan Implementasi Dimensi Track



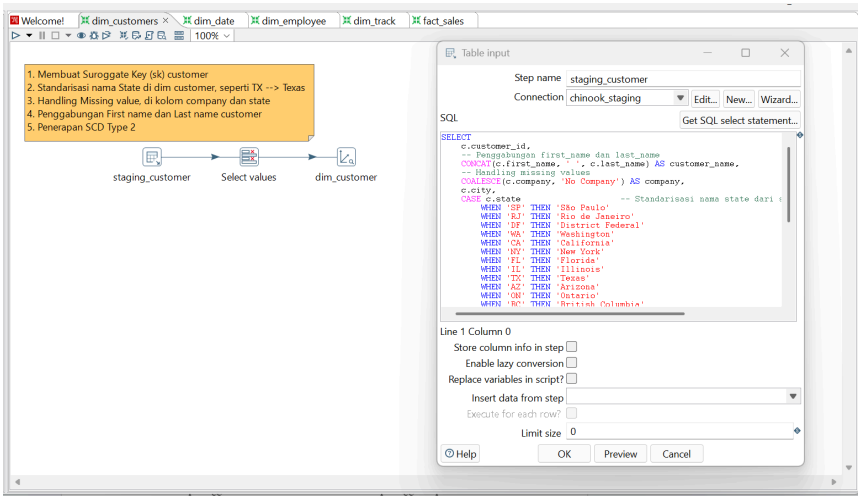

**Gambar 10.** Transformasi Dimensi Track

Tabel *dim_track* dirancang untuk menyimpan seluruh informasi terkait katalog produk musik yang dijual di *Chinook Store*. Pada *dim_track*, kami melakukan beberapa serangkaian proses:

- **Penggabungan Data**
  Pada *database* operasional awal, informasi terkait produk terpecah di beberapa tabel terpisah (*track, album, artist, genre,* dan *media_type*). Pada alur transformasi Pentaho, kami menggabungkan seluruh tabel tersebut menjadi satu bentuk dimensi yang utuh (*dim_track*). Dengan demikian, atribut penting seperti judul album, jenis genre, nama track, dan lainnya langsung siap dianalisis tanpa memerlukan proses pencarian berulang.
- **Transformasi Kategori Durasi Lagu**
  Untuk memberikan nilai tambah bagi analisis bisnis, kolom durasi mentah dalam satuan milidetik (*milliseconds*) diubah menjadi variabel kualitatif dengan nama kolom *duration_category.* Pengkategorian ini dilakukan dengan aturan sebagai berikut:
  - Pendek: durasi kurang dari 3 menit (< 180.000 ms)
  - Standar: durasi antara 3 hingga 5 menit ( 180.000 ms - 300.000 ms)
  - Epik: durasi lebih dari 5 menit (> 300.000 ms)
- **Pembuatan *Surrogate Key***
  Di pentaho, kami menambahkan step bernama *Add Sequence* untuk menghasilkan *Surrogate Key* baru yaitu *sk_track* secara otomatis berurutan mulai dari angka 1. Kunci buatan ini yang berfungsi sebagai *primary key* pada tabel *dim_track* dan nantinya dihubungkan ke tabel fakta.
- **Pengurutan Struktur Kolom**
  Sebelum data dimasukkan ke dalam database tujuan, aliran data dilewatkan ke step *Select Values*. Step ini berfungsi untuk merapikan struktur kolom, memastikan urutan kolom sesuai dengan rancangan awal, serta membuang atribut-atribut mentah yang tidak lagi dibutuhkan oleh tabel *dim_track*.
- **Penanganan Perubahan Data dengan SCD Type 1 (*Step Output, Insert/Update*)**
  Sebagai step penutup (*output*), kami menggunakan step *Insert/Update* untuk menangani pembentukan sekaligus pembaruan data katalog. Step ini menerapkan metode *Slowly Changing Dimension* (SCD) *Type 1* dengan mencocokkan data berdasarkan *track_id*. Jika terdapat data baru, Pentaho akan melakukan perintah *Insert*, sedangkan jika terdapat koreksi atau perubahan informasi lagu pada *database* sumber, data lama pada *dim_track* akan langsung diperbarui. Pendekatan SCD *Type 1* pada *dim_track* ini dipilih karena informasi terkait katalog lagu

bersifat konstan dan perubahan yang terjadi umumnya sebatas perbaikan kekeliruan penulisan pada data.

### 3. Perancangan dan Implementasi Dimensi Customer



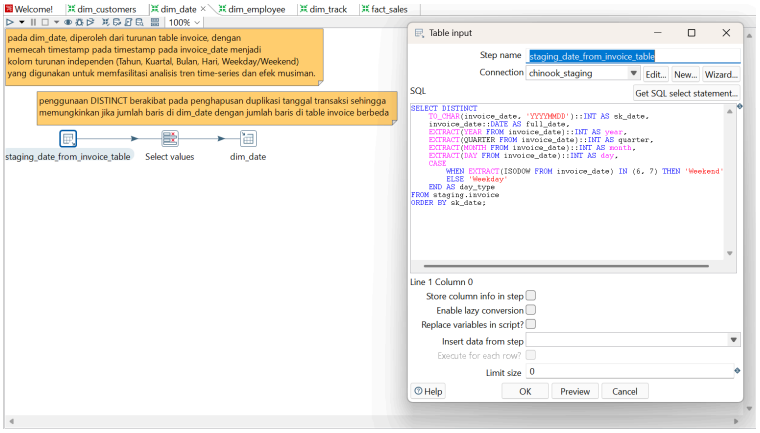

**Gambar 11.** Transformasi Dimensi Customer

Tabel *dim_customer* dirancang untuk menyimpan profil lengkap pelanggan *Chinook Store*. Karena informasi pelanggan seperti alamat domisili sangat penting untuk menganalisis tren penjualan per wilayah pada *Data Mart*. Pada *dim_customer,* kami melakukan serangkaian proses sebagai berikut:

- **Pembersihan Data dan Standarisasi Profil**
  Pada step ini, kami melakukan pembersihan data pelanggan langsung dari tabel *staging*. Pembersihan ini meliputi penggabungan nama pelanggan (kolom *first_name* dan *last_name* digabungkan menjadi satu kolom yang bernama *customer_name*), penanganan *missing values* di kolom *company* (dengan mengisi nilai *null* pada kolom *company* dengan '*No Company*'), serta standarisasi wilayah (kode singkatan wilayah seperti 'TX' diubah menjadi Texas, dan jika data wilayah kosong, maka diisi dengan 'N/A').
- **Pengurutan Struktur Kolom**
  Data yang telah dibersihkan kemudian dilewatkan ke step *Select Values* untuk memilih kolom-kolom relevan (customer_id, customer_name, company, city, state, country, dan email) serta memastikan urutannya konsisten sebelum dikirim ke database tujuan.
- **Penerapan SCD Type 2 dan Pembuatan *Surrogate Key* (*Step Output, Dimension Lookup/Update*)**
  Sebagai step penutup (*output*), kami menggunakan step *Dimension Lookup/Update* untuk mengelola tabel *dim_customer* di *schema dimension.* Step ini bertugas untuk menerapkan metode *Slowly Changing Dimension* (SCD) *Type 2* sekaligus membentuk *Surrogate Key* (*sk_customer*). Berbeda dengan SCD *Type 1* pada *dim_track*, pada

SCD *Type 2* jika terjadi perubahan profil pelanggan, seperti perubahan kota tempat tinggal) maka data lama tidak akan dihapus, tapi akan dibuat baris baru untuk mengisi data baru. Step ini juga secara otomatis mengelola variabel version (nomor urut versi profil), serta kolom rentang waktu (*date_from* dan *date_to*), jadi ketika profil pelanggan diperbarui, maka rekamanan data lama akan ditutup tanggal berlakunya, dan rekaman data baru akan dibuat dengan versi yang lebih tinggi. Pendekatan SCD *Type 2* pada *dim_customer* bertujuan untuk menjaga integritas data historis. Dengan demikian, transaksi masa lalu tetap tercatat sesuai lokasi pelanggan saat transaksi tersebut terjadi, sehingga tidak mengganggu keakuratan laporan analisis penjualan per wilayah di kemudian hari.

## 4. Perancangan dan Implementasi Dimensi Date




**Gambar 12.** Transformasi Dimensi Date

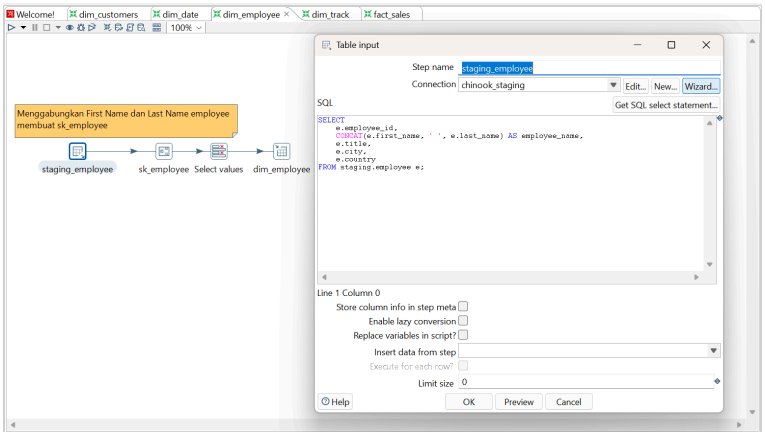
Tabel *dim_date* dirancang sebagai dimensi waktu untuk mendukung analisis tren *time-series*, evaluasi performa penjualan dari waktu ke waktu, serta pemetaan pola musiman (*seasonality*) di *Chinook Store*. Pada *dim_date,* kami melakukan serangkaian proses berikut:

- **Ekstraksi dan Pemecahan Komponen Tanggal (*staging_date_from_invoice_table*)**
  Data tanggal diturunkan langsung dari kolom *invoice_date* pada tabel *invoice*. Pada *query* di *Table Input*, nilai *timestamp* dipecah menjadi atribut-atribut tertentu secara independen. Kolom *sk_date* diformat menjadi angka integer dengan pola YYYYMMDD yang sekaligus berfungsi sebagai *Surrogate Key* untuk disambungkan ke tabel fakta. Kolom *full_date* merupakan tanggal lengkap dengan format YYYY-MM-DD. Lalu kami juga melakukan ekstraksi komponen waktu, dengan memisahkan komponen *year, quarter, month, day*. Setelah itu, kami juga melakukan pengkategorian hari (*day_type*) dengan memilah hari transaksi ke dalam tipe '*Weekend*' (Sabtu dan

Minggu) atau '*Weekday*' (Senin hingga Jumat) untuk melihat pola perilaku pembelian pelanggan di hari kerja versus akhir pekan.

- **Pengeliminasian Duplikasi Tanggal**
  Kami juga mengeliminasi duplikasi tanggal dengan menggunakan query *SELECT DISTINCT*, hal ini menyebabkan setiap tanggal yang terdapat pada tabel *invoice* akan dicatat sebanyak satu kali saja di dalam *dim_date.* Sehingga jumlah baris data di *dim_date* menjadi jauh lebih sedikit dibandingkan dengan total baris pada tabel *invoice*.
- **Pengurutan Struktur Kolom**
  Setelah itu, data-data yang telah unik ini akan dilewatkan ke step *Select Values* untuk memastikan urutan kolom tersusun secara rapi, sebelum dikirim ke *database* target.
- **Penyimpanan ke *Database***
  Sebagai step akhir, kami menggunakan step *Table Output* untuk memuat seluruh baris data *dim_date* langsung ke d*atabase dwh_chinook* bagian *dimension.dim_date*.

## 5. Perancangan dan Implementasi Dimensi Employee




**Gambar 13.** Transformasi Dimensi Employee

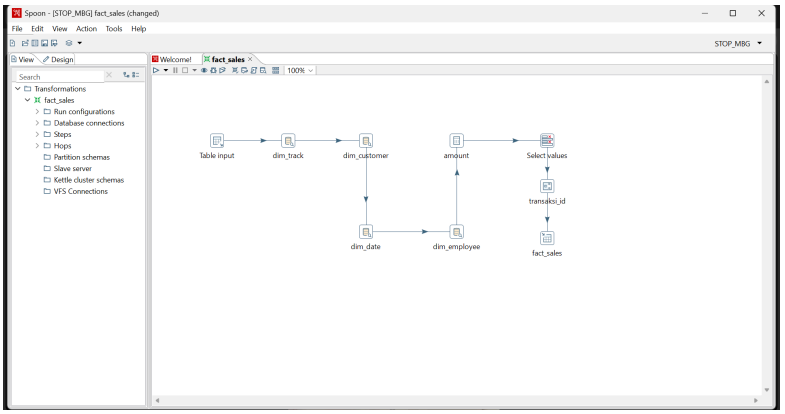
Tabel *dim_employee* dirancang untuk menyimpan informasi terkait staf yang melayani pelanggan di *Chinook Store.* Data ini nantinya akan terhubung ke tabel fakta untuk melihat kontribusi dan performa masing-masing staf penjualan. Pada *dim_employee,* kami melakukan serangkaian proses sebagai berikut:

- **Ekstraksi dan Penggabungan Nama Staf**
  Pada *Table Input*, data staf yang diambil dari tabel *staging.employee,* kami lakukan transformasi sederhana dengan menggabungkan kolom *first_name* dan *last_name* menggunakan fungsi *CONCAT* menjadi satu variabel utuh bernama *employee_name*. Atribut pendukung lainnya seperti *title, city,* dan *country* juga disertakan pada *dim_employee.*
- **Pembuatan *Surrogate Key***

Selanjutnya kami menambahkan step *Add sequence* untuk membentuk *Surrogate Key* yang bernama *sk_employee.* Kunci buatan ini berfungsi sebagai *Primary Key* unik pada tabel *dim_employee*.

- **Pengurutan Struktur Kolom**
  Setelah itu kami merapikan struktur dan urutan kolom pada step *Select Values* agar sesuai dengan rancangan awal kami, yaitu *sk_employee, employee_id, employee_name, title, city*, dan *country*.
- **Penyimpanan ke Database**
  Sebagai step akhir (*output*), kami menggunakan step *Table Output* untuk memuat seluruh baris data dimensi pegawai ke dalam *dimension.dim_employee* di database *dwh_chinook.*

## 6. Perancangan dan Implementasi Tabel Fakta




**Gambar 14**. Transformasi Tabel Fakta (fact_sales)

Tabel *fact_sales* dirancang sebagai tabel utama yang menyimpan informasi transaksi penjualan musik pada *Chinook Store*. Tabel ini berfungsi untuk menghubungkan seluruh tabel dimensi yang telah dibentuk sebelumnya, yaitu *dim_track*, *dim_customer*, *dim_date*, dan *dim_employee*. Melalui tabel fakta ini, kami dapat melakukan analisis penjualan berdasarkan produk musik, pelanggan, waktu transaksi, serta pegawai yang melayani pelanggan. Pada proses pembentukan tabel *fact_sales*, kami melakukan serangkaian proses sebagai berikut:

- **Ekstraksi Data Transaksi**
  Pada tahap awal, kami mengambil data transaksi dari database *staging_chinook*, khususnya tabel *invoice_line*, *invoice*, dan *customer*. Ketiga tabel tersebut digabungkan menggunakan perintah JOIN berdasarkan kolom *invoice_id* dan *customer_id*. Data yang diambil meliputi identitas detail transaksi, identitas pelanggan, track yang dibeli, tanggal transaksi, harga satuan, jumlah pembelian, serta pegawai yang bertanggung jawab melayani pelanggan (*support_rep_id*). Proses ini dilakukan melalui step Table Input pada

Pentaho untuk menghasilkan data transaksi sebagai dasar pembentukan tabel fakta.

- **Pengambilan *Surrogate Key* dari *Tabel Dimensi***
  Selanjutnya Setelah data transaksi diperoleh, kami melakukan pencocokan data dengan masing-masing tabel dimensi menggunakan step Database Lookup. Proses ini bertujuan untuk mendapatkan *Surrogate Key* dari setiap dimensi sebagai pengganti penggunaan kunci asli dari database operasional. Pencocokan dilakukan melalui beberapa tahapan, yaitu:

  1. *dim_track*: mencocokkan *track_id* untuk memperoleh *sk_track*.
  2. *dim_customer*: mencocokkan *customer_id* serta periode berlaku profil pelanggan (*date_from* dan *date_to*) berdasarkan tanggal transaksi untuk memperoleh *sk_customer*.
  3. *dim_date*: mencocokkan tanggal transaksi (*invoice_date*) dengan *full_date* untuk memperoleh *sk_date*.
  4. *dim_employee*: mencocokkan *support_rep_id* dengan *employee_id* untuk memperoleh *sk_employee*.
     Dengan proses tersebut, setiap transaksi dapat dihubungkan dengan informasi pada tabel dimensi yang sesuai. Khusus pada dim_customer, pencocokan periode berlaku diperlukan karena tabel tersebut menggunakan metode SCD Type 2, sehingga informasi pelanggan yang digunakan tetap menyesuaikan kondisi profil pada saat transaksi terjadi.

- **Perhitungan Nilai Penjualan**
  Menggunakan step Calculator untuk membentuk atribut amount yang merepresentasikan nilai penjualan. Pada konfigurasi transformasi, perhitungan dilakukan menggunakan operasi perkalian antara *unit_price* dan *total_quantity*. Hasil perhitungan tersebut digunakan sebagai nilai pendapatan dari setiap baris transaksi yang akan disimpan pada tabel fakta.

- **Pemilihan dan Pengurutan Struktur Kolom**
  Setelah seluruh proses pencocokan dimensi dan perhitungan selesai, data dilewatkan ke step Select Values. Step ini berfungsi untuk memilih atribut yang diperlukan dan merapikan struktur kolom sebelum dimasukkan ke database tujuan. Kolom yang dipertahankan meliputi *sk_track, sk_customer, sk_date, sk_employee, unit_price, quantity*, dan *amount*. Pada tahap ini, kolom *total_quantity* juga diubah namanya menjadi *quantity* agar sesuai dengan struktur tabel fakta yang dirancang.

- **Pembuatan Surrogate Key Tabel Fakta**
  Sebelum data disimpan, kami menambahkan step Sequence untuk menghasilkan identitas unik pada setiap baris tabel fakta dengan nama *transaksi_id*. Sequence dikonfigurasi menggunakan penghitung

internal Pentaho yang dimulai dari angka 1 dan bertambah satu untuk setiap baris data. Kunci ini berfungsi sebagai Primary Key yang membedakan setiap catatan transaksi pada tabel *fact_sales*.

- **Penyimpanan Data ke Database**
  Sebagai tahap akhir, kami menggunakan step Table Output untuk memasukkan seluruh hasil transformasi ke dalam tabel *fact_sales* pada schema *fact_table* di database *dwh_chinook*. Proses pemuatan data menggunakan pengaturan batch insert dengan jumlah commit setiap 1.000 baris. Selain itu, kami mengaktifkan pengaturan truncate table agar data pada tabel tujuan dikosongkan terlebih dahulu sebelum proses pemuatan ulang dilakukan. Dengan demikian, tabel fakta berisi hasil transformasi terbaru dan siap digunakan sebagai dasar pembentukan Data Mart untuk analisis penjualan musik global.

## E. DATA MART AND BUSINESS INSIGHT

### 1. Topik Analisis

Sesuai tujuan proyek ini, yaitu menganalisis penjualan musik global *Chinook Store*, *datamart* yang kami bangun berfokus pada penjualan per genre, negara, dan tahun. Pertanyaan bisnis yang ingin dijawab: genre apa yang paling menghasilkan pendapatan, di negara mana, dan bagaimana perkembangannya dari tahun ke tahun? Hasilnya membantu menentukan genre yang diprioritaskan di katalog dan negara yang layak dibidik pemasaran.

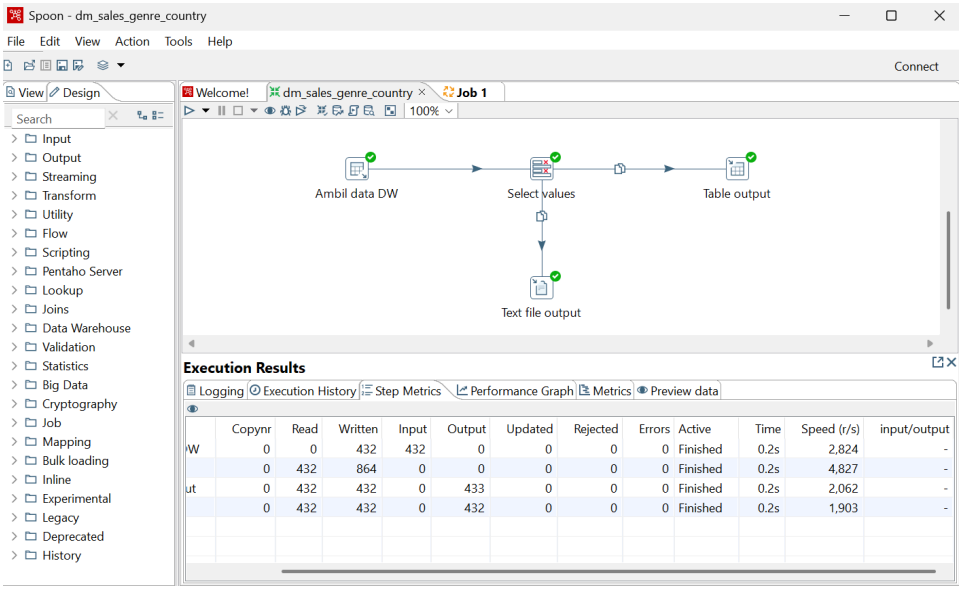
### 2. Desain Datamart

*Datamart* dibangun dari tabel fakta *fact_sales* yang digabung dengan tiga dimensi: *dim_date* (tahun), *dim_customer* (negara), dan *dim_track* (genre). Hasilnya disimpan di tabel *datamart.dm_sales_genre_country* (*PostgreSQL*) dan file *dm_sales_genre_country.csv,* lewat satu transformasi Pentaho.

**Tabel 3.** Struktur Kolom Datamart dm_sales_genre_country

| Kolom | Tipe | Isi |
|---|---|---|
| year | Integer | Tahun transaksi |
| country | Teks | Negara Pelanggan |
| genre_name | Teks | Nama Genre |
| total_revenue | Numerik | Total pendapatan |
| total_units_sold | Integer | Jumlah item terjual |
| customer_count | Integer | Jumlah pelanggan unik pada kombinasi tahun, negara, dan genre |

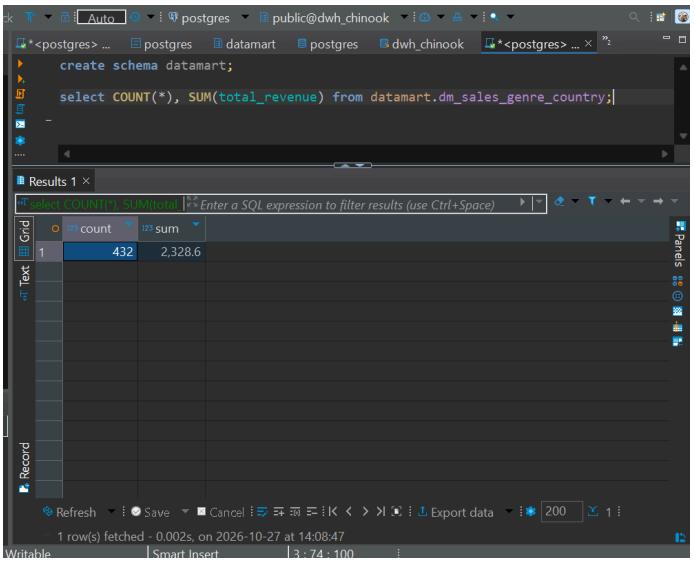
| | | |
|---|---|---|
| pct_of_country_revenue | Numerik | Persentase genre dalam negara dan tahun yang sama |
| revenue_rank | Integer | Peringkat genre dalam negara dan tahun yang sama |




**Gambar 15.** Transformasi dm_sales_genre_country

## 3. Hasil Datamart

*Datamart* berisi 432 baris, mencakup 24 negara, 24 genre, dan 59 pelanggan pada periode 2021–2025. Total pendapatan 2.328,60 dari 2.240 item terjual, sama persis dengan total di data sumber, sehingga tidak ada data yang hilang atau terhitung ganda.




**Gambar 16**. Hasil query tabel datamart

**4. Insight Bisnis**

- **Rock mendominasi, dan empat genre menguasai hampir tiga perempat pendapatan.**

**Tabel 4.** Struktur Kolom Datamart dm_sales_genre_country

| Genre | Pendapatan | Porsi |
|---|---|---|
| Rock | 826,65 | 35,5% |
| Latin | 382,14 | 16,4% |
| Metal | 261,36 | 11,2% |
| Alternaitive & Punk | 241,56 | 10,4% |

Keempatnya menyumbang sekitar 73,5% pendapatan. Rock menjadi genre nomor satu (atau sama kuat) di 22 dari 24 negara. Pengecualiannya Irlandia (TV Shows) dan Swedia (Latin).

- **Pendapatan ditentukan oleh jumlah pelanggan, bukan besar belanja per pelanggan.**

  Rata-rata belanja per pelanggan berkisar di angka 37–47 di hampir semua negara. AS punya 13 pelanggan dengan rata-rata 40,24, sama saja dengan negara yang hanya punya 1 pelanggan. Jadi AS (523,06, 22,5%) dan Kanada (303,96, 13,1%) unggul karena pelanggannya lebih banyak. Lima negara teratas (AS, Kanada, Prancis, Brasil, Jerman) menyumbang sekitar 58,8% pendapatan.

- **Konten video bernilai tinggi per unit.**

  TV Shows menghasilkan 93,53 dari hanya 47 item, sekitar 1,99 per item, sedangkan genre musik rata-rata sekitar 0,99 per item. Karena itu TV Shows (4,0%) berada di peringkat ke-5, di atas Jazz yang terjual 80 item. Pola yang sama terlihat pada Drama, Sci Fi & Fantasy, dan Comedy.

- **Pendapatan stabil, tidak tumbuh.**

  Pendapatan tahunan bergerak di kisaran 450–481 tanpa tren naik yang jelas: 449,46 (2021), 481,45 (2022), 469,58 (2023), 477,53 (2024), dan 450,58 (2025). Jumlah pelanggan aktif per tahun juga nyaris tetap, 46–47.

**5. Rekomendasi Bisnis**

- Pertahankan katalog inti (Rock, Latin, Metal, Alternative & Punk) karena menyumbang sebagian besar pendapatan.

- Perbesar katalog TV Shows dan konten video, karena nilai per unitnya sekitar dua kali lipat musik.
- Fokus pada penambahan pelanggan, terutama di negara yang baru punya satu pelanggan, karena belanja per pelanggan sudah merata.
- Kurangi ketergantungan pada AS dan Kanada (35,6% pendapatan) lewat promosi di Eropa dan Amerika Latin.
- Dorong pertumbuhan, karena pendapatan stagnan selama lima tahun.

**6. Keterbatasan**

- Datanya kecil (59 pelanggan), dan 15 negara hanya punya satu pelanggan. Pola per negara bisa mencerminkan selera satu orang, jadi insight negara sebaiknya dibaca sebagai indikasi, bukan kesimpulan pasti.
- Genre belum distandarkan: “Alternative” terpisah dari “Alternative & Punk”, “Heavy Metal” dari “Metal”, dan “Science Fiction” dari “Sci Fi & Fantasy”. Selisihnya kecil (kurang dari 3% total), tapi bisa sedikit menggeser peringkat.
- customer_count dihitung per kombinasi tahun-negara-genre, jadi tidak boleh dijumlahkan antar baris.
- Tabel fakta tidak memuat invoice_id, sehingga jumlah transaksi tidak bisa dihitung. Ukuran volume yang dipakai adalah jumlah item terjual.
- Pada Argentina, Rock dan Alternative & Punk sama-sama peringkat 1 karena nilainya seri.

## F. CHALLENGES AND LEARNINGS

### 1. Challenges

| Tantangan | Penyebab | Solusi |
|---|---|---|
| Error function round(double precision, integer) does not exist saat preview Table input | Kolom amount bertipe double precision, sedangkan ROUND(x, 2) di PostgreSQL hanya menerima tipe numeric | Hasil agregasi diubah ke numeric dengan ::numeric sebelum dibulatkan |
| Error syntax error at or near "<" padahal query sudah benar | Teks contoh bawaan Pentaho (SELECT <values> FROM <table name> WHERE <conditions>) ikut terkirim karena query ditempel di bawahnya, tanpa menghapusnya | Seluruh isi kotak SQL dihapus (Ctrl+A, Delete), lalu query ditempel ulang dan dicek lewat Preview |
| Tabel datamart “tidak | Editor DBeaver masih | Database |

| | | |
|---|---|---|
| ditemukan” di DBeaver (relation does not exist) | terhubung ke database postgres, bukan dwh_chinook tempat Pentaho menulis hasilnya | dwh_chinook ditampilkan dan dipilih di DBeaver, lalu query dijalankan di sana. Hasilnya 432 baris dengan total 2.328,60 |
| Dua output dari satu step (tabel dan CSV) | Hop kedua dari satu step bisa memakai mode Distribute, yang membagi baris ke dua output | Hop diatur ke Copy data to next steps. Terbukti di Step Metrics: Select values menulis 864 baris (432 × 2), dan masing-masing output menerima 432 baris |
| Penerapan SCD Type 1 di dim_track | Pada awal pengujian step Insert/Update di Pentaho, sistem selalu melakukan pembaruan (*update*) ke seluruh baris data katalog lagu. Padahal, data di tabel *staging* tidak mengalami perubahan, sehingga proses pemrosesan ETL menjadi kurang efisien. | Kami melakukan penyesuaian pada pencocokan kunci (track_id) secara presisi. Kami memisahkan konfigurasi kolom yang hanya diisi saat data baru masuk (*insert*) dan kolom yang diizinkan untuk diperbarui (*update*) jika terjadi koreksi data pada sumber. |
| Penerapan SCD Type 2 di dim_customer | Menyelaraskan struktur DDL di PostgreSQL dengan konfigurasi step Dimension lookup/update di Pentaho dibutuhkan ketelitian. Jika pemetaan version serta rentang tanggal (date_from dan date_to) kurang tepat, rekam data lama pelanggan tidak ditutup secara otomatis, yang mengakibatkan | Kami memastikan skema DDL di PostgreSQL sudah menyiapkan kolom version, date_from dan date_to sesuai spesifikasi Pentaho. Setelah itu, kami melakukan simulasi pembaruan data alamat pelanggan di *staging* untuk memverifikasi bahwa tanggal keberlakuan data lama otomatis tertutup dan baris |

| | | |
|---|---|---|
| | munculnya dua baris data aktif bersamaan. | baru terbentuk dengan versi yang lebih tinggi. |
| Batasan data fakta | fact_sales tidak memuat invoice_id, jadi jumlah transaksi tidak bisa dihitung | Ukuran volume diganti dengan jumlah item terjual (total_units_sold), dan keterbatasan ini dicatat di laporan |
| Error di tabel fakta | Tabel fakta belum menyimpan seluruh id dari setiap dimensi untuk proses database lookup | lakukan join seluruh id yang dibutuhkan pada tabel fakta |

**2. Learnings**

- Error sering datang dari hal kecil. Teks bawaan yang lupa dihapus atau tipe data yang tidak cocok bisa menghentikan seluruh transformation. Membaca bagian Caused by: di log lebih berguna daripada menebak.
- Pastikan database yang dilihat benar. Pentaho dan DBeaver bisa terhubung ke database berbeda. Cek nama database di koneksi sebelum menyimpulkan bahwa data hilang.
- Selalu rekonsiliasi angka. Total datamart (2.328,60) kami cocokkan dengan tabel fakta dan data sumber, sehingga kami yakin tidak ada baris yang hilang atau ganda.
- Hop “Copy” dan “Distribute” berbeda. Untuk satu sumber dengan dua tujuan, hop harus *Copy*.
- Data tidak pernah benar-benar bersih. Nilai kosong, nama kategori yang mirip, dan kolom yang hilang harus diputuskan penanganannya dan dijelaskan, bukan diabaikan.
- Memastikan kolom yang dibutuhkan telah tersedia untuk pencocokan nilai ketika melakukan lookup apapun.
- Penerapan SCD Type 1 terbukti efisien untuk memperbarui data statis tanpa menyimpan riwayat demi menghemat ruang penyimpanan.
- Penerapan SCD Type 2 penting untuk melacak perubahan data pelanggan demi menjaga keakuratan analisis historis pada *Data Mart*.